



Resale Tracker App: Master MERN Curriculum & Build Strategy



Part 1: Screen Architecture Strategy

Your app relies on **4 Core UI Layout Patterns** derived from your iPad sketches:

- **Pattern A (Auth Layout):** Login / Sign Up Switcher.
- **Pattern B (Data Table + Filter Drawer Layout):** All Inventory, Sales, Lookup Lists, Ledger.
- **Pattern C (Dynamic Form / Detail Layout):** Inventory Detail, Bundle Details, Add Bundle Sourcing Form.
- **Pattern D (Dashboard & Metrics Layout):** Home Page, Income Statement, Tax Report.

Build Strategy:

- **Phases 1–3 (HTML/CSS/JS):** Build **4 target screens** (one representing each pattern) to master base styling and DOM logic without writing throwaway code.
- **Phase 4 (React):** Componentize your design system and build out **ALL 19+ screens** across the entire application using client-side routing and state.



Part 2: The Complete MERN Master Roadmap

(August 1, 2026 – March 30, 2027)

[Phase 1: HTML5] → [Phase 2: CSS3] → [Phase 3: JavaScript] → [Phase 4: React] → [Phase 5: Express/Node] → [Phase 6: MongoDB] → [Phase 7: Integration & Desktop]



Phase 1: HTML5 Deep Dive

Timeframe: August 1 – August 21 (3 Weeks)

Primary Learning Source: MDN HTML Guides & freeCodeCamp Responsive Web Design

Concepts to Acquire:

- Document Anatomy (`<!DOCTYPE>`, `<head>`, `<body>`, Meta tags, Viewport).
- Semantic Elements (`<header>`, `<nav>`, `<main>`, `<article>`, `<section>`, `<aside>`, `<footer>`).
- Typography & Text Elements (`<h1>`–`<h6>`, `<p>`, `<span>`, `<strong>`, `<em>`, `<blockquote>`).
- Hyperlinks & Lists (`<a>`, `<ul>`, `<ol>`, `<li>`, `<dl>`).
- Media Elements (`<img>`, `<picture>`, `<svg>`, `<figure>`, `<figcaption>`).
- **Forms (CRITICAL):** `<form>`, `<input>` (types: `text`, `number`, `email`, `password`, `date`, `checkbox`, `radio`), `<select>`, `<option>`, `<textarea>`, `<button>`, `<label>`, fieldsets, attributes (`required`, `placeholder`, `pattern`).

- **Data Display:** `<table>`, `<thead>`, `<tbody>`, `<tfoot>`, `<tr>`, `<th>`, `<td>`, `colspan`, `rowspan`.
- Accessibility (a11y) basics: `aria-*` attributes, image `alt` texts, semantic landmarks.

What You Build (HTML Skeletons):

Code the unstyled HTML structures for **4 key screens**:

1. **Login / Sign Up Toggle View** (Pattern A).
2. **Inventory Table + Filter Modal Overlay** (Pattern B).
3. **Add Bundle / Bundle Details Form** (Pattern C).
4. **Home Page Dashboard** (Pattern D).

Phase 2: Advanced Custom CSS3

Timeframe: August 22 – September 30 (5.5 Weeks)

Primary Learning Source: MDN CSS Guides & CSS-Tricks

Concepts to Acquire:

- **Syntax & Fundamentals:** Rulesets, Selectors (Element, Class `.`, ID `#`, Attribute `[type="text"]`, Pseudo-classes `:hover`, `:focus`, `:nth-child`).
- **Colors & Visuals:** Color models (HEX, RGB/RGBA, HSL), background properties (`background-color`, gradients), borders, `border-radius`, and shadows (`box-shadow`, `text-shadow`).
- **Typography Basics:** Font families (`font-family`), font sizes (`font-size`), line height (`line-height`), text alignment, letter spacing, and Google Fonts integration.
- **The Cascade & Mechanics:** Specificity rules, Inheritance, and global resets (`* { margin: 0; padding: 0; box-sizing: border-box; }`).
- **Box Model:** `margin`, `border`, `padding`, `content`, and `box-sizing: border-box`.
- **Layout Engine 1 (Flexbox):** `display: flex`, directions (`flex-direction`), alignment (`justify-content`, `align-items`), wrapping, flex grow/shrink.
- **Layout Engine 2 (CSS Grid):** `display: grid`, `grid-template-columns`, `fr` units, `minmax()`, `gap`, `auto-fit/fill`.
- **Positioning Modes:** `static`, `relative`, `absolute`, `fixed`, `sticky`, and layer ordering (`z-index`).
- **Modal & Overlay Styling:** Backdrop dimming/blurring (`background: rgba(0, 0, 0, 0.5)`), fixed centering, z-index layering, and sliding side-drawers.
- **CSS Custom Properties (Variables):** Design tokens (`:root { --primary-color: #2e7d32; --bg-main: #f5f5f5; }`) for theme management.
- **Responsive Design:** `@media` queries, mobile-first strategies, fluid units (`vw`, `vh`, `rem`, `em`).

What You Build (Inventory App Design System):

Create a global `styles.css` file and style your **4 target screens**:

1. Style the **Global 2-Column App Shell** (Sidebar Navigation + Top Nav Header).
2. Style the **Login / Sign Up View**.
3. Style the **Inventory Table** with clean column alignments, action buttons, and modal popups.
4. Style the **Home Page Dashboard** (KPI cards, Quick Actions, Aging Listings container).

🟡 Phase 3: Modern JavaScript (ES6+)

Timeframe: October 1 – November 22 (7.5 Weeks)

Primary Learning Source: javascript.info & freeCodeCamp JS Certification

Concepts to Acquire:

- Fundamentals: Variables (`const/let`), Primitives vs Objects, Scope, Hoisting.
- Logic & Functions: Arrow functions (`() => {}`), parameter defaults, ternary operators, `switch` statements.
- **Data Structures & Operations:** Array methods (`.map()`, `.filter()`, `.reduce()`, `.find()`, `.some()`, `.splice()`), Object manipulation, destructuring, Spread/Rest operators (`...`).
- **DOM Manipulation:** `document.querySelector()`, dynamic element creation, updating `innerHTML/textContent`.
- Events: `addEventListener`, event delegation, form `submit` handling, `e.preventDefault()`.
- Persistence: `localStorage` and `sessionStorage` with `JSON.stringify()` / `JSON.parse()`.
- **Asynchronous JS:** Promises, `async/await`, `try/catch` blocks, `fetch()` API for JSON data.

What You Build (Interactive UI Logic):

1. **Login Switcher:** JS class toggle to swap between "Log In" and "Sign Up" forms.
2. **Inventory Filtering Engine:** Filter local inventory by SKU, status, or aging ranges.
3. **Bundle Sourcing Calculator:** Logic to sum item costs, calculate total profit, and toggle confirm popups.
4. **Local Data Persistence:** Save mock inventory data to `localStorage`.

🟣 Phase 4: React Framework (Full App Expansion)

Timeframe: November 23 – January 10, 2027 (7 Weeks)

Primary Learning Source: react.dev (Official Documentation)

Concepts to Acquire:

- Build Tooling: Vite (`npm create vite@latest`), JSX Rules, Component architecture.
- **Props & State:** Passing data via props, managing component state with `useState`.
- Rendering Lists & Conditionals: Mapping arrays with unique `key` props, conditional rendering (`&&`, ternary).
- **Controlled Forms:** Binding form inputs directly to React state.
- **Side Effects (`useEffect`):** Syncing state, fetching data, lifecycle management.
- **React Router (`react-router-dom`):** Client-side routing (`<Routes>`, `<Route>`, `<Link>`, `useNavigate`).
- Context API & Custom Hooks: Global state management across components.

What You Build (ALL 19+ App Screens in React):

Componentize your app (`<Sidebar/>`, `<Table/>`, `<Modal/>`, `<MetricCard/>`) and build **every screen** from your iPad drawings:

- **Auth:** Login / Sign Up Screen.
- **Dashboard:** Home Page Dashboard.
- **Inventory:** All Inventory Table, Inventory Detail Page (with Lifecycle Timeline).
- **Bundles:** Bundles Table, Bundle Details Page, Add Bundle Sourcing Tool.
- **Sales:** Sales Table Page, Sales Detail View.
- **Ledger:** General Expenses, Mileage Tracker, Expense Forms.
- **Lookup Lists:** Categories, Storage Locations, Selling Platforms, Purchase Sources, Suppliers.
- **Insights:** Income Statement View, Tax Report 2026 View.

Phase 5: Backend REST API with Node.js & Express

Timeframe: January 11 – February 14, 2027 (5 Weeks)

Primary Learning Source: Express.js Docs & MDN Server-Side Express/Node

Concepts to Acquire:

- Node.js Runtime: Environment setup, ES Modules (`import/export`), NPM package management.
- **REST API Architecture:** HTTP Methods (`GET`, `POST`, `PUT`, `DELETE`), URL params, query strings, HTTP status codes.
- Express Server setup, routing middleware (`express.json()`), CORS handling (`cors`).
- **Data Validation:** Request body validation using `Zod` or `Joi`.
- Authentication & Security: Password hashing (`bcrypt`) and JSON Web Tokens (`JWT`).

What You Build (Resale Tracker API):

Build Express server endpoints for all resources (`/api/auth`, `/api/inventory`, `/api/bundles`, `/api/sales`, `/api/ledger`, `/api/lookups`).

Phase 6: MongoDB Database & Mongoose

Timeframe: February 15 – March 7, 2027 (3 Weeks)

Primary Learning Source: MongoDB University & Mongoose Docs

Concepts to Acquire:

- NoSQL Principles: Documents, Collections, BSON format vs. Relational SQL databases.
- MongoDB Atlas Setup: Cloud database cluster creation, connection strings, `.env` file management.
- **Mongoose ORM:** Schemas (data types, validation), compiling Models, CRUD operations (`.find()`, `.create()`, `.findByIdAndUpdate()`).
- Schema Relationships: Sub-documents vs Document References (`populate`).

What You Build (Database Architecture):

Define Mongoose Schemas for `User`, `InventoryItem`, `Bundle`, `Sale`, `LedgerEntry`, and `LookupList`, connecting Express controllers directly to MongoDB Atlas.

Phase 7: Full Integration & Desktop Application

Timeframe: March 8 – March 30, 2027 (3 Weeks)

Primary Learning Source: Tauri Documentation

Concepts to Acquire:

- Full-Stack Integration: Fetching backend API data in React with `axios` / `fetch`.
- Production Builds: Optimizing React client code (`npm run build`), environment variables in Vite.
- **Desktop Wrapping:** Packaging web applications into native desktop executables using **Tauri**.

What You Build (Finished Resale Tracker Desktop App):

Wire your React UI to live MongoDB/Express backend endpoints and compile the full app into a native desktop application executable (`.exe` / `.dmg`).